

AITP 2026

ProofJudge

Tool-Grounded LLM Evaluation of
Formal Proof Quality in Mathlib

Shane Caldwell

Dreadnode · shane@dreadnode.io

[arXiv:2608.20432](https://arxiv.org/abs/2608.20432) · github.com/SJCaldwell/ProofJudge · huggingface.co/datasets/SJCaldwell/proofjudge

I'm **Shane Caldwell**, research engineer for **Dreadnode**.

I work on training, evaluation, and monitoring for LLMS acting as security agents.

It is very rare for me to be at a mathematics conference.



dreadnode

AI Infrastructure for the Security Stack

“I think in the future, there will be entire professions of mathematicians who might take a giant Lean-generated proof and do some ablation on it, trying to remove parts of it and find more elegant ways.

*They might get other AIs to do some reinforcement learning to make the proof more elegant, and **maybe other AIs will grade whether this proof looks better or not.**”*

— Terence Tao

THE PROBLEM

Correctness is not enough

I assumed formal
verification made taste
irrelevant. If it compiles,
it's correct.

The mathlib community
disagrees.

The screenshot displays three GitHub pull request (PR) discussions from the `leanprover-community` repository, illustrating the community's stance on LLM-generated code. Each PR is marked as "Closed" with a red icon.

- PR 1:** `feat: add Sophie Germain theorem formalization #38035`. The PR was opened by `kellyzhao1231` and merged into `leanprover-community:master` from `kellyzhao1231:sophie-germain`. A comment by `grunweg` on Apr 22 states: "In this case, while I appreciate your desire to contribute to mathlib, I'm afraid I have to close this PR. The LLM-generated code in this PR fails to meet our standards, and while reviewers and maintainers are willing to help new contributors to improve the quality of their code through the review process, this can only be done if the code is indeed written by somebody who learns in the process, otherwise the quality will not improve in future PRs and the effort required will be larger than the benefit. You're welcome to contribute to mathlib, but this must be fully hand-written code. (For new results, it is also strongly recommended to discuss your contribution on zulip beforehand.)"
- PR 2:** `feat(SimpleGraph): card of common neighbors equals card of triangles containing edge #37109`. The PR was opened by `zeekmartin` and merged into `leanprover-community:master` from `zeekmartin:feat/SimpleGraph-commonNeighbors-triangle`. A comment by `mathlib-triage` (Bot) on Apr 17 states: "We appreciate your desire to contribute to mathlib, but unfortunately we will have to close this PR. The LLM-generated code in this PR fails to meet our standards, and while reviewers and maintainers are willing to help new contributors to improve the quality of their code through the review process, this can only be done if the code is indeed written by somebody who learns in the process, otherwise the quality will not improve in future PRs and the effort required will be larger than the benefit."
- PR 3:** `feat(ContMDiff): add ContMDiff scalar-action typeclasses #36170`. The PR was opened by `SproutSeeds` and merged into `leanprover-community:master` from `SproutSeeds:cody/Issue-5617-single-pass`. A comment by `grunweg` on Apr 22 states: "We appreciate your desire to contribute to mathlib. After internal discussions among the reviews, unfortunately we will have to close this PR. The LLM-generated code in this PR fails to meet our standards, and while reviewers and maintainers are willing to help new contributors to improve the quality of their code through the review process, this can only be done if the code is indeed written by somebody who learns in the process, otherwise the quality will not improve in future PRs and the effort required will be larger than the benefit. You are welcome to contribute to mathlib --- but such code must be hand-written for the first couple of PRs."



plby

Code

Blame

12.5 MB

[View raw](#)

(Sorry about that, but we can't show files that are this big right now.)

▽ Solution.lean ×

▽ Solution.lean

```
248805 infer_instance
248806
248807 theorem mathoverflow_1973 :
248808   ∃ atlas : ChartedSpace (EuclideanSpace C (Fin 3)) (unitSphere 6),
248809     let I := atlas
248810     IsManifold  $\mathcal{I}(C, EuclideanSpace C (Fin 3)) 1 (unitSphere 6) := by
248811       exact SixSphereComplexAtlas.exists_complex_atlas
248812
248813 #print axioms mathoverflow_1973
248814 -- 'Mathoverflow1973.mathoverflow_1973' depends on axioms: [propxet, Class
248815
248816 end Mathoverflow1973
248817
248818 end
248819$ 
```

Proof Generation is scaling faster than proof review

RLVR made the Lean kernel a reward signal.

Anyone with \$20 dollars to pay for a Codex/Claude Code sub can start generating valid proofs.

Mathlib reviewers are a small, scarce, highly skilled group. There is an entire bootcamp devoted to producing more of them.

\$20 dollars for a subscription does not appear to be producing them.

Can LLMs be a productive part of the review process that encourage high quality PRs?

ProofJudge: an agentic judge

*We leave correctness to the Lean Kernel and focus on more **subjective** qualities*

01	02	03	04	05
Library leverage	Automation fit	Structural clarity	Statement quality	Mathlib conventions
Does this reprove something Mathlib already has?	Right tactic for the job	Does the proof decompose, or is it a monolith?	Is the lemma stated generally enough to be reused?	Naming, style, etc

Each dimension scored independently 1–10, the harness combines them into one overall score.

The Agentic Judge Harness

- The judge agent runs in a sandbox holding Mathlib at the commit the PR applies to.
- It greps for existing declarations, reads signatures, and checks how neighbouring lemmas are named
- Budget: **20 tool calls**, then it must commit to a score.

cons_comp_rev mathlib4 #8814

Mathlib/Data/Fin/Tuple/Basic.lean

INITIAL

gpt-5.4-mini-2026-03-17

6 turns · 12 tool calls

45,528 prompt tok

In PROOF UNDER REVIEW

```
theorem cons_comp_rev {α n} (a : α) (f : Fin n → α) :
  Fin.cons a f • Fin.rev = Fin.snoc (f • Fin.rev) a := by
  funext i
  simp only [Function.comp_apply, Function.comp_def]
  induction' f using Fin.consInduction with n b f ih generalizing a
  · funext; simp [Fin.rev, Fin.snoc]
  · simp [Fin.snoc, ih]
    split_ifs with lt_succ_n lt_n
    · have isLt := Nat.sub_lt_self (succ_pos ↑i) (lt_n)
      mfirst! m h
```

29 lines — clipped

1 JUDGE · 1 CALL

pr_diff ()

```
Mathlib/Data/Fin/Basic.lean      | 3 ++
Mathlib/Data/Fin/Tuple/Basic.lean | 30 ++++++
2 files changed, 33 insertions(+)
```

2 JUDGE · 1 CALL

pr_diff (file_path="Mathlib/Data/Fin/Tuple/Basic.lean", offset=1, max_li...

```
index 1065b88d70..1b6c9ba7f5 100644
--- a/Mathlib/Data/Fin/Tuple/Basic.lean
+++ b/Mathlib/Data/Fin/Tuple/Basic.lean
@@ -661,6 +661,36 @@ theorem append_right_cons {n m} {α : Type*} (xs : Fin n →
α) (y : α) (ys :
  Fin.append (Fin.snoc xs y) ys • Fin.cast (Nat.succ_add_eq_succ_add
  ) succ := by
```

3 JUDGE · 1 CALL

find_imports (file_path="Mathlib/Data/Fin/Tuple/Basic.lean")

```
6:import Mathlib.Data.Fin.Basic
7:import Mathlib.Data.Pi.Lex
8:import Mathlib.Data.Set.Intervals.Basic
```

A worked example: PR #11640

`Set.restrictPreimage_isClosedMap`

INITIAL — SENT BACK

```
-- ~20 lines of tactic proof
```

Reconstructs a result Mathlib already provides.



MERGED — ACCEPTED

```
H.restrictPreimage s
```

One line, using the declaration that existed.

WHAT THE JUDGE DID

- Searched the repository for the existing declaration, and found it.
- Quoted the signature back in its reasoning.
- Marked the initial version down specifically on library leverage.

Human preference that already exists

Human judgement baselines are expensive to collect. We get them from GitHub.

- Every reviewed PR has a version sent back and one that was merged. We at least know the final version is better to humans than the initial draft.
- Curated with claude-sonnet-5 to drop lint-only pairs and newly added declarations.
- 218 declarations as part of the test, 123 declarations as part of the dev set

Measuring Alignment With Taste

1

Score

The judge sees one revision and scores it against the rubric.

2

Blind

It never learns another version exists, or what it scored.

3

Compare

Aligned = the merged revision scores higher than the initial one.

4

Strict

Ties and inversions both count as failures.

THE RUN

6

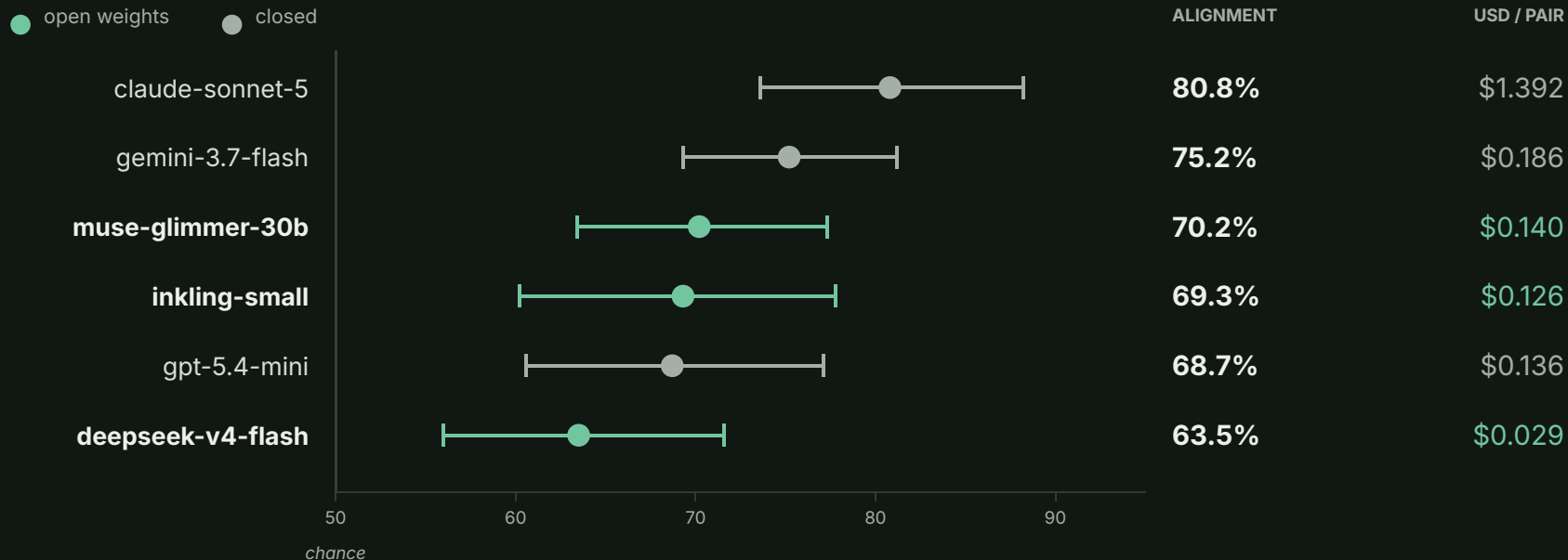
judges — 3 open weight, 3 closed

3

replicates per judge

RESULTS

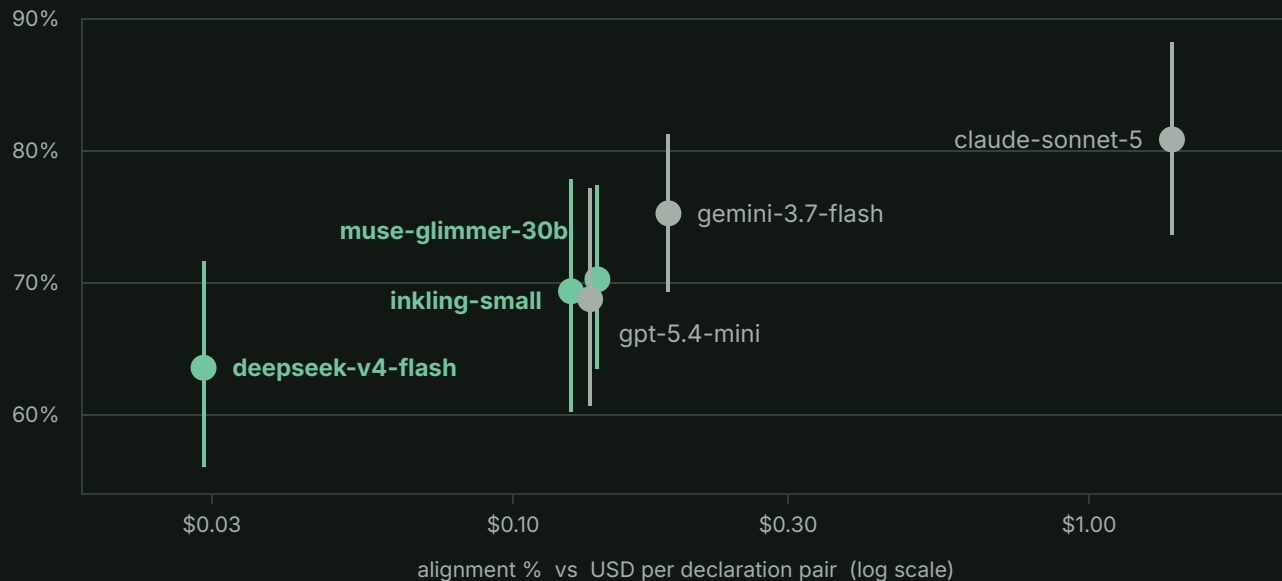
Every judge recovers reviewer preference



COST

Alignment against cost per pair

Cost on a log scale.



muse-glimmer-30b: 70.2% at \$0.140 - a tenth of claude-sonnet-5's \$1.392.

Project Status

Taste is measurable.

Well above chance, with a stable ranking, and an auditable per-dimension breakdown.

Variance is holding back training.

Split the rubric into five single-dimension judges/tool call budget as parameter. Rubric dropout seems to include this as a feature, so maybe it's not a problem.

You don't need frontier models.

Open weights reach ~70% at a tenth of the cost. That's what makes a training loop affordable.

Then: RL against it.

Give a model a signal to iterate a PR against, so it improves before a maintainer ever sees it.

Accepting rubric PRs

github.com/SJCaldwell/ProofJudge
[hf.co/datasets/SJCaldwell/proofjudge](https://huggingface.co/datasets/SJCaldwell/proofjudge)
shane@dreadnode.io

Artifacts

Harness

[github.com/SJCaldwell/
ProofJudge](https://github.com/SJCaldwell/ProofJudge)

Judge agent, sandbox build,
versioned rubrics, analysis scripts.



Dataset

[hf.co/datasets/SJCaldwell/
proofjudge](https://hf.co/datasets/SJCaldwell/proofjudge)

218-pair test split, dev and train
splits, full PR metadata.
Apache-2.0.



Traces

[hf.co/datasets/SJCaldwell/
proofjudge-eval-traces](https://hf.co/datasets/SJCaldwell/proofjudge-eval-traces)

Judge rollouts for auditability.

